

ARCHITECTURE TECHNIQUE

Plateforme intelligente de gestion et de préparation des candidatures aux appels d'offres
assistée par IA

Version : 1.0

Projet : Application web sur mesure — développement from scratch

Utilisateur : un utilisateur principal

Date : Septembre 2026

1. Objectif

Cette architecture traduit l'architecture fonctionnelle en composants logiciels, services, flux de données, technologies, sécurité et déploiement.

Elle vise une solution maintenable, sécurisée et évolutive, adaptée à un projet mono-client sans complexité SaaS inutile.

2. Principes

Architecture modulaire et faiblement couplée.

Séparation Frontend / Backend / traitements asynchrones / IA / stockage.

API REST entre Frontend et Backend.

PostgreSQL comme source principale de vérité.

pgvector pour la recherche sémantique.

Redis pour queue et cache.

Stockage objet pour les fichiers.

Validation humaine pour les décisions et documents critiques.

Abstraction des services externes pour pouvoir les remplacer.

3. Vue d'ensemble

Navigateur → Next.js → FastAPI → services métier → PostgreSQL / Redis / stockage objet → workers → services IA et externes.

Les traitements longs sont exécutés en arrière-plan : recherche, crawling, téléchargement, extraction, embeddings, analyse IA et génération documentaire.

4. Stack technique

Frontend : Next.js + TypeScript + Tailwind CSS.

Backend : Python + FastAPI.

Base : PostgreSQL.

Vector search : pgvector.

Queue/cache : Redis.

Workers : Celery ou Dramatiq.

Stockage : S3 ou Cloudflare R2.

IA : OpenAI API.

Recherche web : WebSearchProvider avec Tavily ou équivalent.

Crawling : Firecrawl et/ou Playwright.

Conteneurisation : Docker.

Déploiement initial : Railway ou infrastructure cloud équivalente.

Monitoring : Sentry.

CI/CD : GitHub Actions.

5. Frontend

Le Frontend fournit l'interface de travail de l'utilisateur unique.

Pages : Dashboard, Profil entreprise, Recherche, Opportunités, Détail tender, Analyse, Éligibilité, Questions, Documents, Génération documentaire, Validation, Historique et Paramètres.

Communication avec FastAPI via REST. Les jobs longs affichent leur état : en attente, en cours, terminé ou échoué.

6. Backend FastAPI

FastAPI gère authentification, validation des requêtes, règles métier, transactions et orchestration.

Organisation recommandée : api/ routes HTTP ; schemas/ Pydantic ; services/ logique métier ; repositories/ accès données ; models/ ORM ; workers/ tâches ; ai/ orchestration IA ; connectors/ services externes ; core/ configuration et sécurité.

7. Traitements asynchrones

Les tâches lourdes ne doivent pas bloquer les requêtes HTTP.

Jobs recommandés : search_tenders, crawl_source, download_tender_documents, extract_document, chunk_document, generate_embeddings, analyze_tender, calculate_match_score, generate_questions, generate_application_documents, build_report.

8. Collecte web

Séparer recherche et extraction : WebSearchProvider → URLs ; CrawlerProvider → contenu ; TenderExtractor → données structurées ; Normalizer → modèle commun ; Deduplicator → doublons.

Une interface commune permet de remplacer un fournisseur sans modifier le métier.

Privilégier les APIs officielles, RSS et données ouvertes lorsque disponibles.

9. Pipeline tender

Recherche → URL → crawling → extraction → normalisation → validation du schéma → déduplication → PostgreSQL → scoring → affichage.

Chaque étape possède un statut et des logs pour diagnostiquer les échecs.

10. Architecture documentaire

Les fichiers originaux sont conservés dans le stockage objet ; PostgreSQL conserve métadonnées et relations.

Pipeline : Upload/Download → stockage → extraction → nettoyage → chunks → embeddings → pgvector → analyse RAG.

Les documents internes et ceux des appels d'offres restent séparables.

11. RAG

Document → chunks → embeddings → pgvector. Pour une requête : embedding → recherche des chunks pertinents → contexte → LLM → réponse structurée.

Chaque chunk conserve des métadonnées : document, page/section si disponible, type et identifiant du tender ou document entreprise.

12. Architecture IA

Services spécialisés : extraction structurée, classification, matching, scoring explicable, questions intelligentes, génération documentaire et résumé.

Les sorties importantes utilisent des schémas structurés.

Prompts, modèles et versions importantes doivent être versionnés.

13. Matching et scoring

Le score combine règles déterministes et analyse sémantique.

Exemple : secteur 20 % ; technologies 15 % ; compétences 15 % ; pays 10 % ; budget 10 % ; expérience 15 % ; certifications 5 % ; éligibilité 10 %.

Conserver les sous-scores et facteurs expliquant le résultat.

14. Génération des principaux documents pour postuler

Entrées : exigences du tender, profil entreprise, documents validés, réponses utilisateur, template et contexte RAG.

Pipeline : template → données validées → sources pertinentes → génération des sections → contrôles → document → validation.

Documents possibles : présentation adaptée, lettre/document de candidature, offre technique, méthodologie, compréhension du besoin, planning, organisation, CV adaptés, références et autres documents requis.

Le système ne doit pas inventer références, certifications, expériences ou données contractuelles.

15. PostgreSQL

PostgreSQL est la source principale de vérité.

Tables principales : users, company, company_profiles, skills, technologies, certifications, experts, projects, references, company_documents, tenders, tender_sources, tender_documents, tender_requirements, tender_criteria, tender_scores, tender_decisions, tender_status_history, questions, question_answers, templates, applications, application_documents, document_versions, notifications, audit_logs.

pgvector est intégré à PostgreSQL pour la recherche sémantique.

16. Stockage fichiers

PDF, DOCX, XLSX et autres fichiers sont stockés dans S3/R2.

PostgreSQL conserve nom, clé, taille, MIME, hash, catégorie, version, expiration, relations et statut.

Le hash aide à détecter les doublons.

17. API REST

Endpoints indicatifs : GET/PUT /company/profile ; POST /searches ; GET /tenders ; GET /tenders/{id} ; POST /tenders/{id}/decision ; POST /tenders/{id}/analyze ; GET /tenders/{id}/requirements ; GET /tenders/{id}/eligibility ; GET/POST /questions ; GET/POST /documents ; POST /applications/{id}/generate ; POST /documents/{id}/validate ; GET /jobs/{id} ; GET /dashboard.

18. Sécurité

Authentification forte même avec un utilisateur principal.

HTTPS en production ; validation des entrées ; contrôle des uploads ; limites de taille/type ; secrets dans variables sécurisées ; logs des opérations sensibles ; principe du moindre privilège.

Protection des documents et des clés API.

19. Données IA et confidentialité

Minimiser les données envoyées aux services IA externes.

Identifier les documents validés comme sources fiables pour la génération.

Conserver une séparation claire entre données de l'entreprise et documents des appels d'offres.

20. Observabilité

Sentry pour les exceptions Frontend/Backend.

Logs structurés pour jobs, sources, téléchargements, extraction et IA.

Métriques : durée des traitements, erreurs par source, documents analysés, temps moyen d'analyse et statut des jobs.

21. CI/CD

GitHub + GitHub Actions pour lint, tests, build Docker et déploiement.

Branches recommandées : main pour production et branches feature/development pour le travail.

Aucune clé secrète dans Git.

22. Environnements

Développement local, staging/test et production.

Variables et secrets séparés par environnement.

Base de données isolée pour staging lorsque possible.

23. Déploiement

Docker standardise l'exécution.

Composants : Frontend, API, Worker(s), PostgreSQL, Redis et stockage objet.

Railway peut héberger les composants applicatifs et la base selon les besoins.

Kubernetes n'est pas nécessaire pour la première version mono-client.

24. Sauvegarde et reprise

Sauvegardes régulières de PostgreSQL.

Stratégie de sauvegarde/versioning des documents.

Procédure de restauration à tester avant la production.

25. Résilience

Timeout, retry contrôlé, backoff et logs pour les appels externes.

L'échec d'une source ne doit pas arrêter toute la recherche.

L'échec d'un document ne doit pas bloquer les autres.

Jobs idempotents lorsque possible pour éviter les doublons lors des retries.

26. Arborescence

- tender-ai/
 - ├─ frontend/
 - ├─ backend/
 - | └─ api/
 - | └─ core/
 - | └─ models/
 - | └─ schemas/
 - | └─ repositories/
 - | └─ services/
 - | └─ ai/
 - | └─ connectors/
 - | └─ workers/
 - ├─ infrastructure/
 - ├─ tests/
 - └─ docs/

27. Flux technique complet

Utilisateur → Next.js → FastAPI → service métier → job Redis → Worker → Connector/Crawler → extraction → PostgreSQL/Stockage → IA/RAG → résultat structuré → PostgreSQL → API → Frontend → validation utilisateur.

Génération : API → job → exigences + données validées + RAG + template → LLM → contrôle → fichier → stockage → validation.

28. Choix recommandés

Commencer avec PostgreSQL + pgvector plutôt qu'une base vectorielle séparée.

Utiliser Redis pour queue/cache et des workers pour les tâches longues.

Conserver des abstractions pour recherche/crawling.

Éviter Kubernetes et une architecture microservices excessive au démarrage.

Prévoir une modularité suffisante pour faire évoluer le système.

29. Étape suivante

Produire ensuite : modèle de données détaillé et ERD ; spécification complète des APIs ; architecture IA détaillée ; contrats des services externes ; structure exacte du code ; stratégie de tests ; Docker Compose local ; stratégie de déploiement production.